

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	V.Pavithra	Reg. No.:	192524274
Date:	24.08.2026	Duration:	60 minutes

Code of Conduct

I _v.pavithra 192524274 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ .pavithrav _____

Annexure A – CI/CD Pipeline Design Exercise

1 INTRODUCTION AND PROBLEM

The Autonomous Electric Vehicle (EV) Charging Network Management System is a distributed, IoT-enabled software platform that manages a network of smart EV charging stations spread across a city or region. The system allows electric vehicle owners to locate available charging stations, reserve charging slots, authenticate and pay for charging sessions, and monitor charging status in real time through a mobile/web application. On the backend, the platform continuously collects telemetry from charging stations (voltage, current, temperature, connector status), performs autonomous load balancing and demand forecasting using machine learning, dynamically prices energy based on grid load, and coordinates with the electricity grid operator for smart demand-response.

Because the system integrates embedded IoT firmware, real-time telemetry pipelines, machine-learning based demand/routing models, payment processing, and customer-facing mobile/web applications, it must be released frequently and safely without disrupting active charging sessions or compromising financial and safety-critical data. A well-designed CI/CD (Continuous Integration / Continuous Deployment) pipeline is therefore essential to automate building, testing, quality analysis, security scanning, and progressive deployment of every code change, while keeping the ability to roll back instantly if a fault is detected in production.

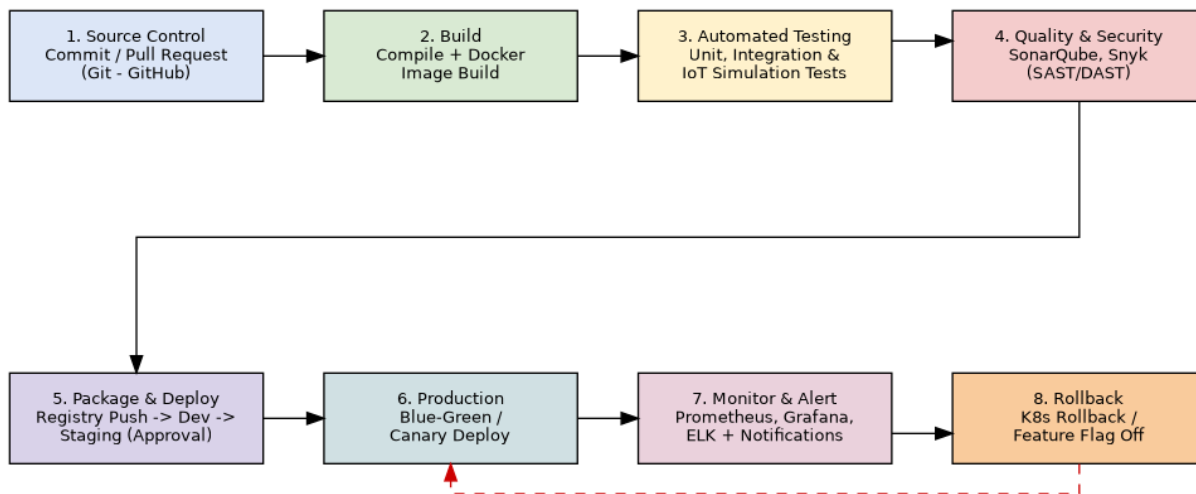
2.REQUIREMENT ANALYSIS

- **Microservices:** Each service can be built, tested, and deployed independently.
- **IoT Integration:** MQTT telemetry and charging-station behavior must be tested using simulations.
- **High Availability:** Zero-downtime deployment and quick rollback are essential for charging safety.
- **Security:** Payment and user data require SAST, DAST, and dependency security checks.
- **Scalability:** Kubernetes enables containers to scale during high demand.
- **Frequent Updates:** Automated CI/CD supports rapid and safe ML and application updates.
- **Multiple Environments:** Dev → Staging → Production allows progressive testing before release.

Overall: The system needs a secure, automated CI/CD pipeline with IoT testing, Kubernetes scalability, progressive deployment, and automatic rollback.

3 ARCHITECTURE

The proposed pipeline consists of eight coordinated stages, moving a code change from developer commit through to monitored production release. The diagram below summarises the end-to-end workflow; a stage-by-stage explanation follows in Section 8.



Pipeline Stages Overview

#	Stage	Purpose
1	Source Code Management	Developers commit to feature branches and raise pull requests; branch protection and mandatory code review trigger the pipeline.
2	Build	Compile each microservice, resolve dependencies, and build versioned Docker images.
3	Automated Testing	Run unit, integration, contract, and IoT/MQTT device-simulation tests.
4	Code Quality & Security Analysis	Static analysis, code-coverage gate, dependency (SCA) and vulnerability (SAST/DAST) scanning.
5	Packaging & Registry Push	Store the immutable, versioned Docker image and Helm chart in a container registry.
6	Deployment (Dev → Staging → Production)	Progressive rollout with automatic deploy to Dev, gated deploy to Staging, and canary/blue-green deploy to Production.
7	Monitoring, Logging & Notification	Real-time observability of application and charging-station health with alerts to the DevOps team.
8	Rollback	Automatic reversal to the last stable release if health checks or key metrics fail post-deployment.

4. Tools and Technology

Pipeline Stage	Recommended Tools	Justification
Source Code Management	Git + GitHub (with branch protection, mandatory PR review)	Industry-standard distributed VCS; GitHub Actions integrates CI directly with repository events, and pull-request gating enforces peer review before merge.
CI Orchestration	GitHub Actions / Jenkins	GitHub Actions provides native trigger-on-commit pipelines with reusable workflows; Jenkins offers flexible plugin-based

Pipeline Stage	Recommended Tools	Justification
		orchestration for on-premise infrastructure often required for grid/IoT integrations.
Build & Containerisation	Maven/Gradle (Java services), npm (Node.js/React app), Docker	Standard build tools per language stack; Docker provides a consistent, portable runtime so that services behave identically across Dev, Staging, and Production.
Unit & Integration Testing	JUnit/Mockito (backend), Jest/React Testing Library (frontend), Postman/Newman (API contract tests)	Widely adopted, well-documented frameworks with strong CI integration and mocking support for isolating microservices during testing.
IoT / Device Simulation Testing	MQTT.js / Eclipse Mosquitto test broker, custom device simulators	Allows the pipeline to simulate hundreds of virtual charging stations publishing telemetry, validating protocol handling without needing physical hardware in CI.
Code Quality Analysis	SonarQube	Provides static analysis, code-smell detection, cyclomatic complexity metrics, and enforceable quality gates (e.g., minimum 80% coverage) before merge.
Security Scanning	Snyk / OWASP Dependency-Check (SCA), OWASP ZAP (DAST)	Detects vulnerable open-source dependencies and runtime vulnerabilities (e.g., injection, auth flaws) — critical given the system handles payments and PII.
Artifact Repository	Docker Hub / Amazon ECR (private registry), Helm chart repository	Secure, versioned storage of immutable build artifacts, enabling reliable rollback to any previous image tag.
Deployment & Orchestration	Kubernetes (K8s) + Helm, Argo CD (GitOps)	Kubernetes gives self-healing, auto-scaling container orchestration suited to variable EV-charging demand; Helm templates environment-specific configuration; Argo

Pipeline Stage	Recommended Tools	Justification
		CD enables declarative, auditable GitOps-based deployment.
Infrastructure as Code	Terraform	Ensures Dev, Staging, and Production infrastructure are reproducible, version-controlled, and consistent, reducing configuration drift.

5 PIPELINE TESTING

- Unit Testing: Tests individual functions/classes; requires $\geq 80\%$ coverage.
- Integration Testing: Checks communication between microservices.
- API Contract Testing: Ensures APIs work correctly without breaking changes.
- IoT Simulation Testing: Simulates charging stations and MQTT telemetry.
- Load Testing: Tests performance under high traffic using JMeter/k6.
- Security Testing: Uses SAST, DAST, and dependency scanning to find vulnerabilities.
- Canary Testing: Sends a small percentage of real traffic to the new version before full deployment.
- Failure Handling: Any failed test stops the pipeline and prevents further deployment.

6. DEVELOPMENT DESIGN

Environment	Purpose	Deployment Trigger	Key Characteristics
Development (Dev)	Rapid feedback for developers on every merged commit	Automatic on merge to `develop` branch	Single replica, synthetic test data, relaxed resource limits, full debug logging.
Staging / Test	Pre-production validation using production-like	Automatic after Dev tests pass; requires	Mirrors production topology at smaller scale; anonymised data; runs full

Environment	Purpose	Deployment Trigger	Key Characteristics
	configuration and simulated device fleet	successful load/security test	regression, performance, and DAST security suite.
Production	Live service for real EV drivers and physical charging stations	Manual approval gate + automated canary rollout	Multi-region Kubernetes cluster, autoscaling, blue-green/canary release, full monitoring and rollback enabled.

Progressive Release Strategy

Production deployment uses a Canary + Blue-Green hybrid strategy to protect active charging sessions:

- **Blue-Green Environments:** A fully separate “Green” environment is deployed alongside the live “Blue” environment; traffic is switched only after the Green environment passes automated smoke tests.
- **Canary Rollout:** Within the Green environment, 5% of station/API traffic is routed to the new version first; if error rate, latency, and charging-success-rate metrics remain within threshold for a defined soak period (e.g., 15 minutes), traffic is progressively increased to 25% → 50% → 100%.
- **Manual Approval Gate:** A release manager must approve promotion from Staging to Production, providing a human checkpoint for safety-critical and financial changes.
- **Zero-Downtime Charging Sessions:** Station-control microservices use graceful connection draining so an in-progress charging session is never interrupted mid-deployment.

7.SECURITY,QUALITY ANALYSIS

7.1 Security Practices

- Store secrets securely in Vault/Kubernetes Secrets, not in code.
- Run SAST, SCA, and DAST security scans automatically.
- Scan and sign container images before deployment.
- Use RBAC to control access to production systems.

7.2 Quality Gates

- Require **≥80% test coverage** and pass SonarQube checks.
- Run automatic lint/code-style checks.
- Require at least **one peer approval** before merging.

7.3 Notifications

- Send CI/CD results to **Slack/Teams**.
- Send production alerts to on-call engineers through **PagerDuty/email**.
- Send daily deployment summaries to leads.

7.4 Rollback Mechanism

- Use versioned Docker images and Helm releases for quick rollback.
- Automatically rollback when health or business metrics fail.
- Use feature flags to disable risky features quickly.
- Keep database migrations backward-compatible.

END TO END BACKEND

- **Commit/PR:** Developer pushes code and creates a PR → CI starts.
- **Build:** Code is compiled and a Docker image is created.
- **Testing:** Unit, integration, API, and IoT tests run automatically.
- **Quality & Security:** SonarQube, Snyk, and OWASP ZAP check quality and security.
- **Package & Push:** After merge, verified images and Helm charts go to the registry.
- **Deployment:** Argo CD deploys to Dev → Staging → Production using canary release.
- **Monitoring:** Prometheus, Grafana, and ELK monitor performance and connectivity.
- **Rollback:** If problems occur, the system automatically rolls back to the last stable version.

CONCLUSION

The proposed CI/CD pipeline automates the complete software delivery lifecycle for the Autonomous EV Charging Network Management System — from source-code commit through build, multi-layered automated testing, code-quality and security scanning, artifact packaging, and progressive Dev → Staging → Production deployment — while embedding continuous monitoring and automated rollback. This

design directly addresses the system's key challenges: safety of live charging sessions, protection of sensitive payment/PII data, high availability under variable demand, and rapid, low-risk iteration of ML-driven demand-forecasting and routing features. By combining industry-standard DevOps tooling (Git/GitHub, Docker, Kubernetes, SonarQube, Prometheus/Grafana) with domain-specific practices such as IoT device simulation testing and canary rollout of charging-station firmware/software updates, the pipeline ensures that new features and fixes reach EV drivers quickly, reliably, and securely.